

AI-ASSISTED RTL VERIFICATION & FAULT DIAGNOSIS

End-to-End VLSI / RTL Verification Project
with Custom Dataset Generation and Machine Learning

Detailed Technical Project Report

Project Component	Implementation
RTL Design	4-bit Arithmetic Logic Unit
Fault Injection	6 intentionally faulty RTL implementations
Verification	SystemVerilog
Simulation	Xilinx Vivado / XSim
Raw Dataset	6,144 verification observations
Behavioral Windows	192 signatures
ML Features	46 behavioral features
Machine Learning	Random Forest
Validation	4-fold Group Cross-Validation
Application	Streamlit Dashboard

Purpose: To demonstrate how RTL verification data can be converted into behavioral signatures and used by machine learning to automatically diagnose the likely underlying RTL implementation fault.

Important scope note: The completed implementation does not contain an IoT subsystem. The final project is an AI-assisted RTL verification and fault-diagnosis system with a Streamlit software dashboard.

Table of Contents

Chapter	Topic
1	Project Abstract
2	Introduction
3	Problem Statement
4	Project Objectives
5	Technology Stack
6	Overall System Architecture
7	4-bit ALU Fundamentals
8	Golden RTL Design
9	Fault Injection
10	Detailed Fault Classes
11	SystemVerilog Verification
12	Vivado/XSim Simulation
13	Custom Dataset Generation
14	Raw Dataset Structure
15	Dataset Statistics
16	Behavioral Windows
17	Behavioral Signatures
18	Feature Engineering
19	The 46 ML Features
20	Why Machine Learning?
21	ML Model Experiments
22	Random Forest
23	Training
24	Group Cross-Validation
25	Hyperparameter Tuning
26	Feature Importance
27	Misclassification Analysis
28	Hierarchical Diagnosis
29	Final Model
30	Real-Time Diagnosis
31	Streamlit Dashboard

Chapter	Topic
32	IoT Clarification
33	Complete End-to-End Workflow
34	Results
35	Limitations
36	Future Improvements
37	Interview Explanation
38	Conclusion

1. Project Abstract

RTL verification is one of the most important stages in digital IC and ASIC development. A verification environment can identify whether the observed output of an RTL implementation matches the expected behavior. However, a traditional pass/fail result does not necessarily explain which implementation fault produced the failure.

This project extends RTL verification by adding a machine-learning-based fault diagnosis layer. A 4-bit ALU is used as the base RTL design. Six intentionally faulty versions of the ALU are created. A SystemVerilog verification environment executes the designs in Xilinx Vivado/XSim and compares actual outputs with golden expected outputs.

Instead of using an external machine-learning dataset, the project creates its own dataset directly from RTL simulation results. The verification environment produces 6,144 raw observations. These observations are then grouped into 192 behavioral windows and converted into 46 numerical behavioral features.

Several machine-learning models are evaluated, including Decision Tree, Random Forest and SVM. Group-based validation is used to prevent vectors belonging to the same behavioral window from appearing in both training and testing data.

The final system uses a Random Forest classifier. The trained model receives a behavioral signature and predicts the most likely RTL fault class. A Streamlit application provides an interactive interface for displaying the diagnosis.

The major idea of the project is therefore: use the behavioral fingerprint produced by RTL verification as the input to an AI-based fault-diagnosis system.

2. Introduction

2.1 What is RTL?

RTL stands for Register Transfer Level. It is a hardware description abstraction used to describe how digital information moves between registers and how combinational logic transforms that information.

Languages such as Verilog and SystemVerilog are commonly used to describe RTL. The RTL description can later be synthesized into gates and implemented on an FPGA or fabricated as part of an ASIC.

2.2 What is RTL Verification?

RTL verification checks whether the hardware implementation behaves according to its intended specification. A verification environment generates inputs, applies them to the design under test, observes outputs and compares those outputs against expected results.

For this project, the expected behavior is generated from the golden 4-bit ALU specification. The faulty implementations are then tested against this reference.

2.3 Why Fault Diagnosis?

Suppose a testbench detects that an ALU produces an incorrect output. The verification engineer knows that something is wrong, but a failure alone does not necessarily identify the root cause.

Different RTL bugs can produce different patterns of errors. One fault may affect only carry during addition, while another may affect multiple result bits during AND operations. These patterns can be treated as behavioral signatures.

The project investigates whether machine learning can recognize those signatures automatically.

2.4 Core Research Question

Can RTL verification behavior be converted into a feature vector that allows a machine-learning model to identify the underlying injected RTL fault?

3. Problem Statement

Traditional verification primarily answers the question: **Does the RTL implementation behave correctly?**

This project adds another question:

What type of RTL implementation fault is most likely responsible for the observed behavioral failure pattern?

The problem is formulated as a supervised classification task.

Element	Meaning
Input	Behavioral signature containing 46 features
Output	Fault class from 1 to 6
Classes	Six intentionally injected RTL faults
Training data	192 generated behavioral signatures
Classifier	Random Forest

3.1 Why Use a Custom Dataset?

A major feature of this project is that the dataset is not taken from a generic machine-learning repository. The data must describe specific RTL behavior, so it is generated directly from the verification environment.

This makes the dataset highly relevant to the actual hardware problem. Every row corresponds to a real simulation observation and every target label corresponds to the RTL fault that generated that observation.

3.2 Supervised Learning Formulation

For supervised learning, the training data contains both the input behavioral measurements and the known fault label. During training, the Random Forest learns statistical relationships between the features and fault classes.

During diagnosis, the fault label is not supplied to the classifier. Only the behavioral signature is supplied.

4. Project Objectives

1. Design a simple reference 4-bit ALU in RTL.
2. Create multiple faulty RTL implementations using controlled fault injection.
3. Develop a SystemVerilog verification environment capable of exercising the ALU.
4. Compare expected and actual result/carry behavior.
5. Generate a custom verification dataset automatically.
6. Group vector-level observations into behavioral windows.
7. Generate behavioral signatures from each window.
8. Extract 46 meaningful ML features.
9. Train and compare multiple machine-learning classifiers.
10. Evaluate models using group-based cross-validation.
11. Analyze feature importance and fault confusion.
12. Tune the Random Forest model.
13. Train a final model using the complete signature dataset.
14. Build a diagnosis engine that predicts an RTL fault from a behavioral window.
15. Expose the final system through a Streamlit dashboard.

4.1 Learning Objectives

The project also provides practical experience across multiple engineering domains: digital logic, Verilog/SystemVerilog, RTL verification, simulation, Python data processing, machine learning, model evaluation, and application development.

This makes the project interdisciplinary rather than being only an RTL design project or only a machine-learning project.

5. Technology Stack

Technology	Role in Project
Verilog	RTL implementation of ALU and faulty variants
SystemVerilog	Verification testbench and dataset generation
Vivado	RTL project and simulation environment
XSim	Behavioral simulation
Python	Dataset analysis, ML and diagnosis
Pandas	Dataset manipulation and feature processing
Scikit-learn	Machine-learning models
Joblib	Saving/loading trained Random Forest
Streamlit	Interactive dashboard
CSV	Dataset and ML-result storage
Git/GitHub	Project version control and repository hosting

5.1 Software Architecture

The project is divided into several logical layers.

Layer	Purpose
RTL Layer	Golden ALU and faulty ALU implementations
Verification Layer	SystemVerilog testbench
Simulation Layer	Vivado/XSim execution
Dataset Layer	Raw vectors and behavioral signatures
ML Layer	Feature engineering and classification
Diagnosis Layer	Runtime fault prediction
Application Layer	Streamlit dashboard

5.2 Project Directory

```
AI_RTL_Verification/
|
|-- rtl/
|   |-- alu.v
|   |-- alu_faulty.v
|   |-- alu_fault2.v
|   |-- alu_fault3.v
|   |-- alu_fault4.v
|   |-- alu_fault5.v
|   |-- alu_fault6.v
|
|-- tb/
|   |-- new_verification_tb.sv
|
|-- dataset/
|   |-- fault_dataset.csv
|   |-- fault_signatures.csv
|   |-- ml_features.csv
```



```
| -- ml/
| | -- feature_engineering.py
| | -- build_fault_signatures.py
| | -- cross_validate_model.py
| | -- tune_signature_model.py
| | -- final_train_model.py
| | -- real_time_diagnosis.py
|
| -- outputs/
| | -- final_model/
| | -- signature_models/
|
| -- app.py
| -- requirements.txt
| -- README.md
```

6. Overall System Architecture

The complete project can be understood as a sequence of transformations.

Step 1: RTL is created.

Step 2: Controlled faults are injected.

Step 3: SystemVerilog generates verification vectors.

Step 4: Vivado/XSim simulates the designs.

Step 5: Expected and actual outputs are compared.

Step 6: Raw observations are stored as a dataset.

Step 7: Observations are grouped into behavioral windows.

Step 8: Each window is converted into a 46-feature behavioral signature.

Step 9: Random Forest learns fault patterns.

Step 10: New behavioral signatures are classified.

Step 11: Streamlit displays the diagnosis.

Conceptually:

Hardware behavior → verification evidence → numerical representation → machine learning → fault diagnosis

6.1 Why This Architecture Is Useful

The important engineering principle is that the AI layer does not directly modify or replace the RTL verification environment. The verification environment remains responsible for determining actual versus expected behavior.

Machine learning is placed above the verification layer. Its job is to interpret the accumulated behavioral evidence and estimate which known fault class best explains that evidence.

7. 4-bit ALU Fundamentals

The Arithmetic Logic Unit is the central hardware design used for the project. A 4-bit ALU is small enough to make simulation and analysis manageable while still providing multiple independent operations and meaningful fault patterns.

7.1 Inputs

Signal	Width	Description
A	4 bits	First operand
B	4 bits	Second operand
OP	2 bits	Operation selector

7.2 Outputs

Signal	Width	Description
RESULT	4 bits	ALU operation result
CARRY	1 bit	Carry generated during addition

7.3 Operation Table

OP	Operation	Equation
00	ADD	$RESULT = A + B$
01	AND	$RESULT = A \& B$
10	OR	$RESULT = A B$
11	XOR	$RESULT = A \wedge B$

7.4 Addition Example

For example, if $A = 15$ and $B = 12$:

$15 + 12 = 27$
27 decimal = 11011 binary

RESULT = 1011
CARRY = 1

The 4-bit result contains the lower four bits while the fifth bit becomes the carry output.

7.5 Why the ALU Is Suitable

The four operations produce different behavioral patterns. Therefore, a fault affecting one operation can be detected by selectively exercising that operation and comparing the observed output with the golden reference.

8. Golden RTL Design

The golden ALU represents the expected correct behavior. It is used as the reference against which the faulty implementations are evaluated.

The verification environment effectively performs:

```
expected_result = Golden_ALU(A, B, OP)
actual_result = Faulty_ALU(A, B, OP)
result_error = (expected_result != actual_result)
carry_error = (expected_carry != actual_carry)
```

8.1 Golden Reference Concept

The golden model is important because machine learning cannot determine whether an output is correct by itself. The verification system first establishes what the correct output should be.

The AI layer therefore depends on the quality of the verification evidence.

8.2 Expected vs Actual

Condition	Interpretation
Expected = Actual	Vector passes
Expected ≠ Actual	Result error
Expected carry ≠ Actual carry	Carry error
Multiple result bits differ	Higher bit-error count

8.3 Bit-Level Comparison

The project also calculates how many individual result bits differ. This is more informative than a simple pass/fail flag because two faulty outputs can differ by one bit or by all four bits.

This bit-level information becomes one of the important behavioral signals used by the ML model.

9. Fault Injection

Fault injection means deliberately modifying an otherwise correct RTL implementation so that it behaves incorrectly in a controlled and known way.

In this project, fault injection is essential because the machine-learning dataset requires known target labels. If a simulation produces a failure and we know which injected fault generated it, the observation can be used as a supervised training example.

9.1 Fault Injection Process

1. Create a correct golden ALU.
2. Duplicate the design into faulty versions.
3. Introduce one intentional implementation error per faulty module.
4. Compile all variants with the verification environment.
5. Apply identical test behavior to every variant.
6. Record expected and actual results.
7. Assign the corresponding `fault_id`.

9.2 Why Multiple Faults?

A single fault would only create a binary classification problem. Six faults make the problem more realistic because the classifier must distinguish between several behavioral patterns.

9.3 Fault Classes

ID	Fault
1	ADD Carry Fault
2	AND-to-OR Logic Fault
3	XOR-to-XNOR Logic Fault
4	OR-to-XOR Logic Fault
5	Opcode Selection Fault
6	Inverted B Operand Fault

10. Detailed Fault Classes

10.1 Fault #1 — ADD Carry Fault

The first faulty implementation intentionally prevents the ADD operation from producing the correct carry output.

The result may still be numerically correct for many vectors, but the carry output becomes incorrect when the addition generates a carry.

This creates a behavioral signature dominated by carry mismatches rather than result-bit mismatches.

10.2 Fault #2 — AND-to-OR Logic Fault

The AND operation is intentionally replaced with OR behavior.

Correct:
 $RESULT = A \& B$

Faulty:
 $RESULT = A | B$

Whenever $A \& B$ differs from $A | B$, the result becomes incorrect. This produces result and bit-level errors concentrated around AND tests.

10.3 Fault #3 — XOR-to-XNOR Logic Fault

The XOR operation is intentionally replaced by its complement, XNOR.

Correct:
 $RESULT = A \wedge B$

Faulty:
 $RESULT = \sim(A \wedge B)$

Because XNOR is the inverse of XOR for each result bit, this fault can produce strong bit-level error signatures.

10.4 Fault #4 — OR-to-XOR Logic Fault

The OR operation is intentionally implemented as XOR.

Correct:
 $RESULT = A | B$

Faulty:
 $RESULT = A \wedge B$

The two operations are identical for some input combinations but differ when both operands contain a logic 1 in the same bit position. This explains why the fault does not necessarily fail every OR vector.

10.5 Fault #5 — Opcode Selection Fault

The fifth fault changes the operation-selection logic. Instead of mapping opcode 10 to OR, the faulty implementation selects AND.

```
Correct:
OP = 10 -> OR
```

```
Faulty:
OP = 10 -> AND
```

This is different from simply changing an operator because the problem is located in the control/opcode-selection path.

10.6 Fault #6 — Inverted B Operand Fault

The sixth fault uses the inverted value of B for the AND operation.

```
Correct:
RESULT = A & B
```

```
Faulty:
RESULT = A & ~B
```

This creates a characteristic pattern in which the AND operation produces incorrect result bits based on the complement of B.

10.7 Why the Faults Can Be Confused

Some faults produce overlapping signatures. For example, Fault #2 and Fault #6 both affect AND behavior. Fault #4 and Fault #5 can also produce related OR-operation error patterns.

This is one of the most important reasons why the project uses an entire behavioral window instead of attempting diagnosis from one test vector.

11. SystemVerilog Verification

SystemVerilog is used to construct the verification environment around the faulty ALU implementations.

11.1 Verification Responsibilities

- Generate operand combinations.
- Select the ALU operation.
- Calculate the expected result.
- Calculate the expected carry.
- Observe the actual result.
- Observe the actual carry.
- Compare expected and actual behavior.
- Calculate bit-level differences.
- Store the verification information.

11.2 Result Error

The result error is a binary indicator:

```
result_error = 1
if expected_result != actual_result
```

11.3 Carry Error

Similarly:

```
carry_error = 1
if expected_carry != actual_carry
```

11.4 Bit Errors

The result bits are compared individually. For a 4-bit result, the number of mismatching bit positions ranges from zero to four.

This gives the ML pipeline more information than a simple result_error flag.

11.5 Dataset Generation

The verification testbench was extended beyond ordinary checking. It generates the structured behavioral dataset needed by the Python ML stage.

12. Vivado / XSim Simulation

Xilinx Vivado is used as the hardware-development and simulation environment. XSim executes the SystemVerilog verification environment.

12.1 Simulation Flow

```
RTL files
|
v
SystemVerilog Testbench
|
v
Vivado Compilation
|
v
XSim Elaboration
|
v
Behavioral Simulation
|
v
Verification Results
|
v
CSV Dataset
```

12.2 Final Dataset Generation Result

Parameter	Value
Fault classes	6
Behavioral windows	32
Vectors per window	32
Vectors per fault	1024
Total CSV rows	6144

The final simulation successfully generated the dataset file:

```
dataset/fault_dataset.csv
```

This file becomes the bridge between the hardware-verification environment and the Python machine-learning environment.

12.3 Why This Bridge Matters

The project is not simply a Python ML project using arbitrary numbers. The numbers originate from actual RTL simulation behavior.

Therefore the ML model is trained on hardware-verification evidence.

13. Custom Dataset Generation

One of the strongest parts of this project is the creation of a custom dataset specifically for RTL fault diagnosis.

13.1 Dataset Creation Formula

6 fault classes × 32 behavioral windows × 32 vectors = 6,144 raw observations

13.2 Why 1,024 Rows per Fault?

Each fault is tested over 32 behavioral windows. Each window contains 32 vectors. Therefore:

$32 \times 32 = 1,024$ observations per fault.

With six fault classes:

$1,024 \times 6 = 6,144$ total observations.

13.3 Balanced Classes

Fault ID	Observations
1	1,024
2	1,024
3	1,024
4	1,024
5	1,024
6	1,024
TOTAL	6,144

The balanced distribution prevents one fault class from dominating the raw dataset simply because it has more observations.

14. Raw Dataset Structure

The raw dataset contains 11 columns.

Column	Description
fault_id	Known injected fault class
A	First 4-bit operand
B	Second 4-bit operand
OP	2-bit operation code
expected_result	Golden result
expected_carry	Golden carry
actual_result	Observed faulty result
actual_carry	Observed faulty carry
result_error	Result mismatch flag
carry_error	Carry mismatch flag
bit_errors	Number of incorrect result bits

14.1 Example Row

fault_id	A	B	OP	expected_result	expected_carry	actual_result	actual_carry	result_error	carry_error	bit_errors
1	0	0	0	0	0	0	0	0	0	0

The first rows can contain no errors because some input combinations do not activate the injected fault. This is expected and is actually important for learning the behavioral pattern.

14.2 Why Passing Vectors Matter

A faulty design does not necessarily fail every test. The combination of passing and failing vectors is itself part of the behavioral fingerprint.

15. Dataset Statistics

The final dataset contains:

Metric	Value
Rows	6,144
Columns	11
Fault classes	6
Rows per fault	1,024
Operations	4
Rows per operation	1,536
Missing values	0

15.1 Error Statistics

Fault	Result Errors	Carry Errors	Bit Errors
1	0	56	0
2	240	0	512
3	256	0	1024
4	148	0	192
5	240	0	512
6	256	0	640

15.2 Interpretation

Fault #1 is primarily a carry-level fault. Its result error count and bit-error count are zero in the analyzed dataset, while carry errors occur.

Fault #3 has the largest bit-error total because XOR-to-XNOR changes the logic value of the result bits.

Faults #2 and #6 both affect AND behavior, which helps explain why they can be more difficult to distinguish using certain features.

Fault #4 produces fewer errors than several other logic faults because OR and XOR produce the same output for some input patterns.

16. Behavioral Windows

The raw dataset operates at the individual-vector level. However, diagnosis is more useful when a group of vectors is considered together.

16.1 Window Definition

A behavioral window contains 32 vectors.

Each fault class contains 32 windows.

Therefore:

6 faults × 32 windows = 192 behavioral signatures.

16.2 Why Windows?

Consider two different faults that happen to produce the same output for one particular input. A single vector cannot reliably distinguish them.

If 32 carefully generated vectors are considered together, the overall error distribution becomes more informative.

16.3 Window-Level Features

The window aggregation calculates statistics such as:

- Total number of result errors
- Total number of carry errors
- Total number of bit errors
- Number of error vectors
- Overall error rate
- Average bit errors
- Maximum bit errors
- Operation-specific errors
- Result-bit error counts
- Expected and actual carry counts

16.4 Behavioral Fingerprint

The resulting collection of statistics can be viewed as the behavioral fingerprint of the RTL fault.

17. Behavioral Signatures

The file **fault_signatures.csv** contains the aggregated behavioral representation of the raw verification data.

The dataset contains 192 rows and 49 columns.

Quantity	Value
Behavioral signatures	192
Total signature columns	49
Excluded columns	3
Final ML features	46

17.1 Signature Structure

Each signature corresponds to one fault/window combination.

For example, a signature might represent:

```
Fault #1
Window #20
32 verification vectors
|
v
Aggregated behavioral statistics
|
v
46-dimensional feature vector
```

17.2 Signature vs Raw Dataset

Raw Dataset	Behavioral Signature
6,144 rows	192 rows
Individual vectors	32 vectors aggregated
11 columns	49 columns
Detailed observations	Statistical fingerprint

This dimensional abstraction makes the machine-learning problem much more meaningful for diagnosis.

18. Feature Engineering

Feature engineering converts the behavioral signature into numerical variables that can be supplied to the machine-learning classifier.

18.1 Why Feature Engineering?

Machine-learning algorithms work on numerical representations. The raw RTL verification data contains useful information, but the information must be structured so that the classifier can recognize patterns.

18.2 Excluded Columns

Column	Reason for exclusion
fault_id	This is the target label.
window_id	Identifier, not behavioral evidence.
num_vectors	Constant/structural information rather than fault identity.

Therefore:

49 total columns – 3 excluded columns = 46 ML features.

18.3 Feature Categories

Category	Examples
Global errors	total_result_errors, total_bit_errors
Rates	error_rate
Bit statistics	avg_bit_errors, max_bit_errors
Operand statistics	avg_A, avg_B, A_ones_total
ADD behavior	ADD_result_errors, ADD_carry_errors
AND behavior	AND_result_errors, AND_bit_errors
OR behavior	OR_result_errors, OR_bit_errors
XOR behavior	XOR_result_errors, XOR_bit_errors
Result-bit behavior	s0_error_count ... s3_error_count
Carry behavior	expected_carry_count, actual_carry_count

19. The 46 ML Features

No.	Feature
1	total_result_errors
2	total_carry_errors
3	total_bit_errors
4	error_vectors
5	error_rate
6	avg_bit_errors
7	max_bit_errors
8	avg_A
9	avg_B
10	avg_OP
11	A_ones_total
12	B_ones_total
13	ADD_tests
14	ADD_result_errors
15	ADD_carry_errors
16	ADD_bit_errors
17	ADD_error_rate
18	ADD_avg_bit_errors
19	ADD_max_bit_errors
20	AND_tests
21	AND_result_errors
22	AND_carry_errors
23	AND_bit_errors
24	AND_error_rate
25	AND_avg_bit_errors
26	AND_max_bit_errors
27	OR_tests
28	OR_result_errors
29	OR_carry_errors
30	OR_bit_errors
31	OR_error_rate

No.	Feature
32	OR_avg_bit_errors
33	OR_max_bit_errors
34	XOR_tests
35	XOR_result_errors
36	XOR_carry_errors
37	XOR_bit_errors
38	XOR_error_rate
39	XOR_avg_bit_errors
40	XOR_max_bit_errors
41	s0_error_count
42	s1_error_count
43	s2_error_count
44	s3_error_count
45	expected_carry_count
46	actual_carry_count

These 46 features form the complete input representation used by the final Random Forest classifier.

20. Why Machine Learning?

A deterministic verification environment can tell us that the behavior is incorrect. However, once multiple possible faults exist, automatically mapping an observed error pattern to the most likely fault becomes a classification problem.

20.1 Conventional Rule-Based Diagnosis

A simple approach would be to write rules such as:

```
IF carry errors occur only during ADD
THEN Fault #1

IF AND operation produces OR-like behavior
THEN Fault #2

IF XOR result is inverted
THEN Fault #3
```

This can work for a small demonstration, but it becomes difficult to maintain as the number of faults and behavioral features increases.

20.2 ML-Based Diagnosis

Machine learning provides a statistical approach. Instead of manually writing every rule, the model learns relationships from labeled examples.

The classifier receives the 46-feature signature and estimates the probability of each fault class.

20.3 Important Principle

AI does not replace verification.

The verification environment still generates the evidence. AI interprets that evidence.

21. Machine Learning Model Experiments

The project experimented with multiple classification algorithms before selecting the final Random Forest baseline.

Model	Initial signature split accuracy
Decision Tree	100.00%
Random Forest	100.00%
SVM RBF	100.00%

The 100% result from the initial split should not be interpreted as proof of perfect real-world generalization because related behavioral windows can make random train/test splitting optimistic.

Therefore, the project introduced group-based cross-validation.

21.1 Why Group Validation?

The 192 signatures are organized into 32 behavioral windows. Samples from the same window should not be allowed to leak into both training and testing.

Group validation keeps complete behavioral windows together.

21.2 Models Tested

- Decision Tree
- Random Forest
- SVM with RBF kernel
- Random Forest variants during hyperparameter tuning
- Hierarchical global + specialist architecture

After the experiments, the global Random Forest remained the preferred architecture.

22. Random Forest

Random Forest is an ensemble machine-learning algorithm based on multiple decision trees.

22.1 Basic Idea

Instead of relying on a single decision tree, Random Forest trains many trees using different subsets of the training data and feature space.

Each tree makes a prediction. The forest combines those predictions to produce the final class.

22.2 Why It Fits This Project

- The dataset is relatively small.
- The features are numerical.
- Behavioral relationships may be nonlinear.
- Different features interact with each other.
- Feature importance can be extracted.
- It does not require the features to be normally distributed.

22.3 Final Configuration

Parameter	Final value
Algorithm	Random Forest
Trees	500
Input features	46
Classes	6
Training signatures	192

22.4 Output

The model predicts one of six classes:

```
FAULT #1
FAULT #2
FAULT #3
FAULT #4
FAULT #5
FAULT #6
```

23. Model Training

The final training stage uses the complete set of 192 behavioral signatures.

The target column is **fault_id**. The remaining 46 selected columns are used as model inputs.

23.1 Training Process

1. Load `fault_signatures.csv`.
2. Separate `fault_id` from the features.
3. Remove `window_id` and `num_vectors`.
4. Construct the 46-feature matrix.
5. Create the Random Forest classifier.
6. Train using all 192 signatures.
7. Save the trained model.
8. Save the feature list.
9. Save model metadata.
10. Save feature importance.

23.2 Generated Model Files

```
outputs/final_model/  
|  
|-- final_random_forest.pkl  
|-- final_feature_list.csv  
|-- model_metadata.csv  
|-- final_feature_importance.csv
```

23.3 Why Save the Feature List?

The feature list guarantees that runtime diagnosis uses the same feature ordering expected by the trained model.

A machine-learning model expects its input columns in the same semantic order used during training.

24. 4-Fold Group Cross-Validation

The project uses four group-based folds to obtain a stronger estimate of generalization to unseen behavioral windows.

Fold	Accuracy
Fold 1	66.67%
Fold 2	100.00%
Fold 3	100.00%
Fold 4	91.67%
Mean	89.58%
Standard Deviation	13.66%

24.1 Interpretation

Fold 1 is the most difficult fold. The model confused Fault #2 with Fault #6 and Fault #4 with Fault #1.

Fold 2 was classified perfectly.

Fold 3 was also classified perfectly after the final feature/model setup.

Fold 4 contained confusion between Fault #4 and Fault #5.

24.2 Why the Accuracy Is Not 100%

The faults do not all produce completely unique behavioral fingerprints. Some fault classes share operation-level characteristics.

For example, Fault #2 and Fault #6 both influence AND behavior. Therefore, their signatures can overlap.

This is actually an important result because it shows why diagnosis is non-trivial rather than merely memorizing a unique label.

25. Random Forest Hyperparameter Tuning

Several Random Forest configurations were tested.

Model	Mean Accuracy	Std Dev
RF_500_sqrt	88.54%	12.97%
RF_1000_sqrt	88.54%	12.97%
RF_500_log2	88.54%	12.97%
RF_500_all	89.58%	13.66%
RF_500_balanced	89.58%	13.66%
RF_1000_balanced	89.58%	13.66%

25.1 Best Tested Model

RF_500_balanced was selected as the best tested configuration with a mean group-validation accuracy of **89.58%**.

25.2 Important Observation

Increasing the number of trees from 500 to 1,000 did not improve the measured group-validation accuracy in the tested configurations.

This demonstrates an important engineering principle: increasing model complexity does not automatically produce better generalization.

26. Feature Importance

Random Forest provides a feature-importance measure that indicates which behavioral variables contributed most strongly to the classification.

Rank	Feature	Importance
1	s3_error_count	0.116268
2	avg_bit_errors	0.071322
3	total_bit_errors	0.069553
4	AND_bit_errors	0.044728
5	AND_max_bit_errors	0.042742
6	AND_avg_bit_errors	0.041932
7	max_bit_errors	0.041759
8	s2_error_count	0.041626
9	OR_bit_errors	0.041487
10	OR_avg_bit_errors	0.041328

26.1 What Does This Mean?

The model relies heavily on observable error behavior. Bit-level error counts are particularly informative.

For example, s3_error_count tells the model how frequently the most significant result bit differs from the expected behavior.

The model therefore does not need to inspect the RTL source code during classification. It uses the behavioral consequences of the fault.

26.2 Engineering Interpretation

This is conceptually similar to a fingerprint. Different faults leave different behavioral fingerprints in the verification results.

27. Misclassification Analysis

Misclassification analysis was performed to understand exactly where the Random Forest struggled.

27.1 Global Misclassifications

Actual Fault	Predicted Fault	Count
2	6	8
4	1	8
4	5	4

There were 20 misclassified samples in the analyzed cross-validation results.

27.2 Fault #2 vs Fault #6

The analysis found that the strongest separator between Fault #2 and Fault #6 was s3_error_count.

Feature	Fault #2 mean	Fault #6 mean
s3_error_count	4.0	8.0
AND_max_bit_errors	2.9375	4.0
total_bit_errors	16.0	20.0
avg_bit_errors	0.500	0.625

The signatures are similar in several other dimensions, which explains why the classifier can confuse these two faults.

27.3 Fault #4 vs Fault #5

Fault #4 and Fault #5 can also overlap because both produce OR-related behavioral failures.

The strongest separating features include s3_error_count, OR_avg_bit_errors, total_bit_errors, OR_bit_errors and OR_result_errors.

28. Hierarchical Diagnosis Experiment

An advanced idea was also tested: instead of using one global classifier, a global Random Forest would first identify a likely fault region and specialist classifiers would then distinguish between known confusing pairs.

28.1 Specialist Pairs

- Fault #2 vs Fault #6
- Fault #4 vs Fault #1
- Fault #4 vs Fault #5

28.2 Result

Fold	Global RF	Hierarchical	Improvement
1	66.67%	66.67%	+0.00%
2	100.00%	100.00%	+0.00%
3	100.00%	66.67%	-33.33%
4	91.67%	91.67%	+0.00%
Mean	89.58%	81.25%	-8.33%

The hierarchical approach reduced overall performance.

Therefore, the project correctly rejected this architecture rather than keeping it simply because it sounded more advanced.

Final decision: retain the tuned global Random Forest.

29. Final Model

The final training stage uses all 192 behavioral signatures to train the production/demo Random Forest model.

Parameter	Value
Training samples	192
Features	46
Fault classes	6
Trees	500
Model type	Random Forest

29.1 Saved Model

The final model is stored as:

```
outputs/final_model/final_random_forest.pkl
```

29.2 Supporting Files

```
final_feature_list.csv  
model_metadata.csv  
final_feature_importance.csv
```

29.3 Why a Saved Model Is Useful

The model can be loaded later without retraining. This is important for the diagnosis application because the user should not have to train the model every time the dashboard starts.

30. Real-Time Diagnosis Engine

The real-time diagnosis script represents the inference stage of the project.

30.1 Runtime Flow

```
Load trained Random Forest
|
v
Load feature list
|
v
Load verification dataset
|
v
Select behavioral window
|
v
Generate 46-feature vector
|
v
Model prediction
|
v
Probability distribution
|
v
Fault diagnosis
```

30.2 Example Diagnosis

A demonstration using Fault #1 and a behavioral window produced:

Output	Result
Predicted Fault	FAULT #1
Fault Name	ADD Carry Fault
Affected Operation	ADD
Confidence	100.00%
Result Errors	0
Carry Errors	2
Bit Errors	0

The important point is that the diagnosis is based on the behavioral signature rather than directly passing the `fault_id` to the classifier.

31. Streamlit Dashboard

The Streamlit application is the user-facing layer of the project.

31.1 Why Streamlit?

The ML model itself is a Python object. A command-line script is useful during development, but an interactive dashboard makes the system easier to demonstrate.

31.2 Dashboard Responsibilities

- Present project information.
- Expose behavioral data for inspection.
- Allow diagnosis interaction.
- Display predicted fault.
- Display confidence/probability.
- Display behavioral error statistics.
- Present fault-analysis information in a human-readable way.

31.3 Running the Application

```
python -m streamlit run app.py
```

The application is then available through the local Streamlit server.

The dashboard should be understood as the **application layer** around the RTL verification and ML pipeline.

31.4 What Streamlit Does NOT Do

Streamlit does not simulate the RTL. Vivado/XSim performs the simulation.

Streamlit does not train the model every time. The trained Random Forest is loaded from the saved model file.

32. IoT Clarification

The completed Project 2 implementation does **not** contain an IoT subsystem.

This is important to document correctly rather than artificially adding an IoT description that does not exist in the implementation.

32.1 What Would Count as IoT?

An IoT implementation would normally involve some combination of a physical or connected device, sensors, communication, telemetry, a network protocol, cloud/backend processing, or remote monitoring.

32.2 What This Project Actually Contains

Component	Present?
RTL hardware design	Yes
SystemVerilog verification	Yes
Vivado/XSim simulation	Yes
Custom dataset generation	Yes
Machine learning	Yes
Fault diagnosis	Yes
Streamlit dashboard	Yes
Physical sensors	No
MQTT/IoT telemetry	No
Cloud IoT platform	No
Microcontroller IoT node	No

Therefore, the correct technical classification is:

AI-Assisted RTL Verification and Fault Diagnosis

33. Complete End-to-End Working

The complete project can now be understood as one continuous chain.

Stage 1 — RTL

A 4-bit ALU defines the expected digital behavior.

Stage 2 — Fault Injection

Six intentionally faulty versions introduce known implementation bugs.

Stage 3 — Verification

SystemVerilog generates inputs and compares actual behavior against the golden reference.

Stage 4 — Simulation

Vivado/XSim executes the verification environment.

Stage 5 — Raw Dataset

The testbench records 6,144 observations containing operands, operations, expected outputs, actual outputs and error information.

Stage 6 — Behavioral Signatures

The raw vectors are grouped into 192 behavioral windows.

Stage 7 — Feature Engineering

Each behavioral window is converted into 46 ML features.

Stage 8 — Machine Learning

Random Forest learns the mapping from behavioral signature to fault class.

Stage 9 — Validation

Group cross-validation estimates how well the model generalizes to unseen behavioral windows.

Stage 10 — Diagnosis

A new behavioral signature is passed to the saved model.

Stage 11 — Application

Streamlit displays the diagnosis and supporting behavioral information.

34. Final Results

Metric	Final Result
Fault classes	6
Raw observations	6,144
Rows per fault	1,024
Behavioral windows	32
Behavioral signatures	192
Signature columns	49
ML features	46
Final model	Random Forest
Trees	500
Group CV mean	89.58%
Group CV standard deviation	13.66%

34.1 Fault-Wise Hierarchical Results

Fault	Hierarchical accuracy
Fault #1	100.00%
Fault #2	75.00%
Fault #3	100.00%
Fault #4	37.50%
Fault #5	75.00%
Fault #6	100.00%

34.2 Main Result

The final selected architecture is the global Random Forest because the hierarchical alternative reduced mean accuracy from 89.58% to 81.25%.

The project therefore demonstrates not only successful model training but also model selection based on measured validation performance.

35. Limitations

The project is a controlled proof-of-concept rather than a complete industrial ASIC verification platform.

35.1 Limited Fault Set

Only six fault classes are currently represented.

35.2 Small Signature Dataset

There are 192 behavioral signatures. This is sufficient for demonstrating the methodology but is small compared with industrial ML datasets.

35.3 Controlled ALU

The design under test is a 4-bit ALU. Larger RTL blocks could produce much more complicated behavioral signatures.

35.4 Synthetic/Controlled Dataset

The data is generated from intentionally injected faults. Real silicon failures may exhibit different characteristics.

35.5 Generalization

The 89.58% group-validation result should not be interpreted as a universal accuracy figure for arbitrary RTL bugs.

35.6 No IoT Layer

The project does not currently provide hardware telemetry or cloud-based remote monitoring.

36. Future Improvements

36.1 More RTL Blocks

The methodology could be extended from a 4-bit ALU to larger blocks such as counters, UARTs, FIFOs, bus interfaces, processors or arithmetic units.

36.2 More Fault Types

Additional faults could include stuck-at faults, timing-related faults, incorrect state transitions, reset faults, width mismatches and control logic errors.

36.3 Larger Dataset

Thousands or millions of behavioral signatures could improve statistical robustness.

36.4 Deep Learning

Sequence models or neural networks could potentially operate directly on longer verification traces instead of manually aggregated features.

36.5 Explainable AI

SHAP or similar methods could explain individual fault predictions.

36.6 Automated CI/CD Verification

The system could be integrated into a Git-based RTL verification pipeline where new simulation failures automatically trigger diagnosis.

36.7 Real Hardware Integration

A future version could connect FPGA test results to the diagnosis engine.

36.8 IoT Extension

If an IoT requirement is desired in a future version, a connected FPGA or embedded device could stream verification/diagnostic telemetry to a server or dashboard.

37. Interview Explanation

37.1 30-Second Explanation

I built an AI-assisted RTL verification system for a 4-bit ALU. I intentionally injected six different RTL faults and created a SystemVerilog verification environment in Vivado/XSim. The testbench generated 6,144 verification observations, which I converted into 192 behavioral signatures with 46 ML features. I trained and validated Random Forest models to classify the behavioral signatures into six fault classes. The final model achieved 89.58% mean accuracy under group-based cross-validation, and I integrated the diagnosis system into a Streamlit dashboard.

37.2 What Did AI Actually Do?

The AI did not generate the RTL and it did not replace the verification environment. It learned the relationship between behavioral failure patterns and known RTL fault classes.

37.3 How Did You Create the Dataset?

I created it myself from RTL simulation. I had six faulty ALU designs. The SystemVerilog testbench applied the verification vectors, calculated expected behavior, captured actual behavior and stored the differences. This produced 6,144 raw observations.

37.4 Why 46 Features?

The behavioral signature dataset had 49 columns. `fault_id` was the target, while `window_id` and `num_vectors` were excluded from the ML input. That left 46 behavioral features.

37.5 Why Random Forest?

Random Forest performed well on the structured numerical features, supports nonlinear relationships and provides feature importance. The tuned global Random Forest was also better than the tested hierarchical specialist architecture.

37.6 What Was the Most Difficult Part?

A major challenge was distinguishing faults that produce similar behavioral patterns. Fault #2 and Fault #6, for example, both affect AND behavior. That is why window-level aggregation and group validation were important.

38. Conclusion

This project demonstrates a complete connection between digital hardware verification and machine learning.

The project begins with a simple 4-bit ALU. Controlled RTL faults are introduced to create known failure modes. A SystemVerilog verification environment running in Vivado/XSim exercises those designs and compares their outputs against golden behavior.

Instead of discarding the verification results after determining pass/fail, the project transforms them into a structured dataset. The final dataset contains 6,144 raw observations.

These observations are aggregated into 192 behavioral signatures. Feature engineering converts each signature into 46 numerical behavioral features representing global errors, operation-specific behavior, bit-level errors, carry behavior and other statistics.

Machine learning is then used to learn the relationship between these behavioral fingerprints and the six known RTL fault classes.

Multiple algorithms were evaluated. Group-based validation showed that the final Random Forest achieved a mean accuracy of 89.58% with a standard deviation of 13.66%.

The project also demonstrates an important engineering practice: not every more complicated model is automatically better. The hierarchical diagnosis architecture reduced performance and was therefore rejected.

The final trained Random Forest is integrated into a diagnosis engine and a Streamlit dashboard, completing the path from RTL simulation to AI-assisted diagnosis.

In one sentence:

The project converts RTL verification failures into behavioral fingerprints and uses machine learning to identify the most likely underlying RTL fault.

Final Project Identity

Category	Final Description
Project type	AI-Assisted RTL Verification
Hardware domain	VLSI / Digital Design
Design under test	4-bit ALU
Verification	SystemVerilog + Vivado/XSim
Dataset	Custom dataset generated from simulation
AI/ML	Random Forest fault classification
Application	Streamlit diagnosis dashboard
IoT	Not included in current implementation

END OF REPORT